

Campus Event Management System — Requirements

1. Customer Statement of Requirements

The Campus Event Management System is a web-based platform designed to centralize the discovery, promotion, and management of events hosted on a university campus. Currently, students rely on fragmented channels — flyers, social media, and email — to learn about campus events, while student organizations lack an efficient way to publicize events and track attendance. This application will allow authenticated students to browse and register for events, enable organizers to create and manage event listings, and provide administrators with oversight to approve events and monitor overall platform activity. The system aims to increase student engagement with campus life while reducing the administrative burden on event organizers. The scope of this application covers event creation, discovery, registration, and attendance tracking; it does not include payment processing, venue conflict resolution, or integration with external calendar systems in its initial version.

2. Requirements Specification

Functional Requirements

1. Users can register for an account and log in (Student, Organizer, Admin roles).
2. Organizers can create, edit, and delete event listings (title, description, date/time, location, capacity, category).
3. Students can browse, search, and filter events by date, category, or keyword.
4. Students can register (RSVP) for an event.
5. Students can view a list of events they have registered for.
6. Organizers can view a list of registered attendees for their events.
7. Admins can approve or reject newly submitted events before they go public.
8. Admins can manage user accounts (view, disable, delete).
9. The system sends a confirmation notification when a student registers for an event.

10. Organizers can mark attendance at an event (check-in).

Non-Functional Requirements

1. **Performance:** Pages should load within 2 seconds under normal load.
2. **Scalability:** The system should support at least 5,000 concurrent users during peak registration periods (e.g., start of semester).
3. **Security:** Passwords must be hashed; role-based access control must restrict actions by user type.
4. **Usability:** The interface should be navigable by a first-time user without instructions, and be mobile-responsive.
5. **Availability:** The system should maintain 99% uptime during the academic term.
6. **Maintainability:** Code should be modular to allow future features (e.g., payments) to be added with minimal rework.
7. **Data Integrity:** Event capacity limits must be enforced to prevent overbooking.

3. Data and Storage Blueprint

Data Input

- Manual entry via web forms for account creation, event creation, and RSVP actions.
- Organizers manually input event details (title, description, date, time, capacity).
- Optionally, administrators may bulk-import events via CSV upload (secondary/nice-to-have feature).

Database / Storage

Choice: PostgreSQL (SQL database)

Justification: Event data is highly relational — users, events, registrations, and attendance records all reference one another with clear foreign-key relationships (e.g., a registration links a user to an event). A relational database enforces this structural integrity and supports queries such as "all events a student registered for" or "all attendees for an event" efficiently. PostgreSQL also supports transactional constraint enforcement (e.g., preventing overbooking via capacity checks).

Alternative considered: NoSQL (MongoDB) — rejected because the core relationships between users, events, and registrations would require significant manual consistency handling in a document model.

4. Agile Product Backlog

The following user stories are tracked as individual Issues in the GitHub repository's Issues tab.

1. As a **student**, I want to **create an account and log in**, so that **I can access personalized event features**.
2. As an **organizer**, I want to **create a new event listing**, so that **students can discover and register for it**.
3. As an **organizer**, I want to **edit or delete my event listings**, so that **I can keep event information accurate**.
4. As a **student**, I want to **search and filter events by date or category**, so that **I can quickly find events relevant to me**.
5. As a **student**, I want to **register for an event**, so that **I can secure my spot and receive updates**.
6. As a **student**, I want to **view my list of registered events**, so that **I can keep track of my schedule**.
7. As an **organizer**, I want to **view a list of attendees for my event**, so that **I can plan logistics and communicate with registrants**.
8. As an **admin**, I want to **approve or reject submitted events**, so that **only appropriate content is published**.
9. As an **admin**, I want to **manage user accounts**, so that **I can maintain platform integrity**.
10. As a **student**, I want to **receive a confirmation notification after registering**, so that **I know my registration was successful**.
11. As an **organizer**, I want to **check in attendees at the event**, so that **I can track actual attendance versus RSVPs**.